从“会查数”到“懂业务”
——奇虎360企业级DataAgent的语义层演进与量化评测体系

演讲嘉宾：郭朝阳

目录

CONTENT

01 背景与痛点

02 NL2SQL范式转变、技术选型

03 双路径混合路由

04 持续完善的评测体系

01 背景与痛点

业务背景

奇麟湖仓

01

数十万张表

日常运维的企业级数仓规模

- 用户抵触语义层建设
- 无从下手
- 不了了之

02

上百万字段

口径散落其中的字段资产

hive iceberg paimon
archer starrocks mysql

03

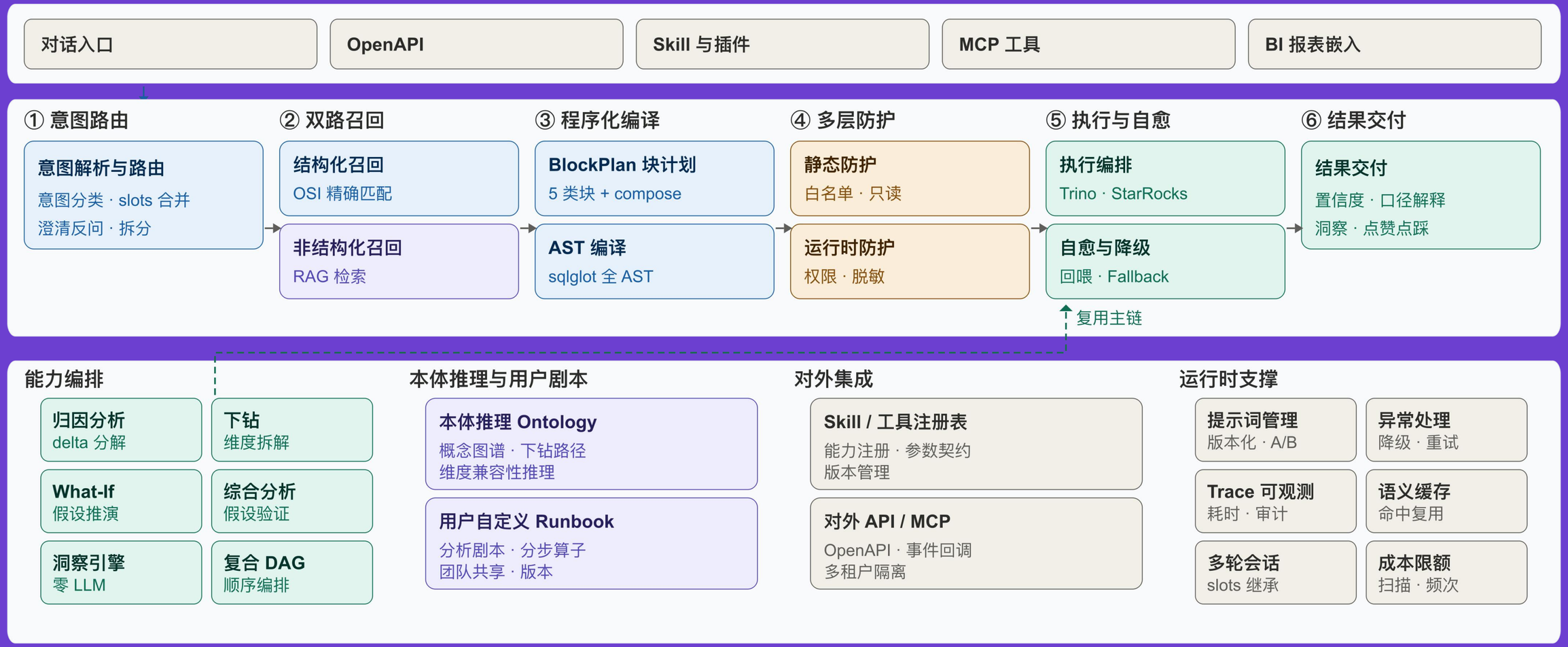
数十条核心业务线

支撑业务运转的数据底盘

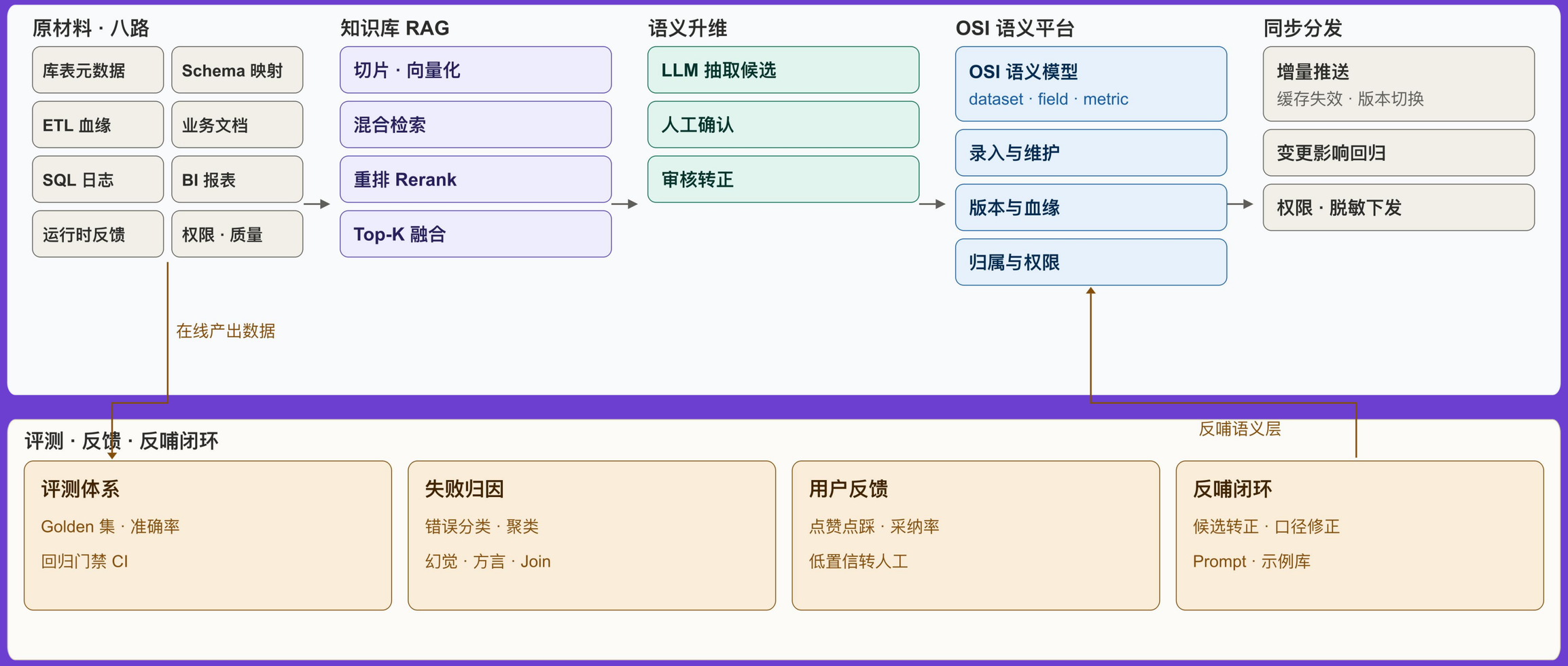
Agent 支撑业务：大数据集群运维 · S3/PoleFS 存储运维 · Ops 机房运维 · 文库 · 会员中台等

NL2SQL → 语义层冷启动 → Agent 能力体系 → 本体化转型

奇麟Agent 架构概览



语义层治理与评测反哺闭环



02 NL2SQL范式转变、技术选型

按模型自由度递减、前置语义约束递增排列



- 往上走会发生什么：模型要猜的东西越来越少，幻觉空间越来越小。
- 代价是什么：你要提前建模的东西越来越多，冷启动越来越贵。

模型从「猜映射」变成「做选择」——列名、JOIN、口径的翻译被外置给人工建模的语义层 + 确定性编译器，模型的输出只剩封闭集合里的引用

物理细节外置程度，并说明不同系统实际自由度不同。白名单只能拒绝未知引用，不能拒绝〝合法但选错的指标

职责和抽象程度的比较，不是严格互斥的产业分类

命名混乱的根源

中间层叫什么（DSL / MQL / LF / LP / Semantic Query / SMQ）其实不重要，真正决定系统能力的是三件事

② 有没有确定性编译器兜底

① 中间层的表达上界（能表达哪些查询结构）

③ 语义建模的人工成本。用名字吵架是浪费时间



	谁生成的	什么时候	内容
语义层定义（地图）	人工（越来越多用 LLM 辅助起草，但人工审核确认才生效）	离线，长期维护	业务对象、关系、事件、时间锚点、权限、指标口径
查询实例（订单）	模型	用户提问的那一刻	“我要指标 X、按维度 Y、时间 Z”——一个引用封闭集合的 JSON

演进：按业务成熟度分层分治

中心化语义仓
资产持续沉淀

用户需求天然分布在一条光谱上

不要求治理 —— 本体级严苛

今天的应对：按业务成熟度分层分治

L1 字段级
DSL：表 · 列直查
独立提示词与编译路径

L2 指标级
MQL：指标引用
口径进入独立编译路径

L3 本体级
LF：实体 · 属性 · 关系
独立提示词与编译路径

三套运行时并行维护
Prompt · Validator · Compiler · Regression

层级是业务平均值
查询命中的是具体元素

缺口风险不对称
指标自拼聚合偏差 87%

语义越完善，边界越交错
跨格式与多轮上下文断裂

分档初衷是让物料少的业务也能用，却把业务平均属性带进了单次查询

错误想法：

“L3 编译器不能编译 L1 语义”；

“LF 天生比 DSL 有更强表达能力”；

“统一契约后所有缺口都可以自动补齐”；

“业务是 L3，所以本次查询一定具备 L3 保障”

语义能力取决于实际登记的物料和编译器算子能力，而不是契约记法本身。统一契约解决的是运行时分裂问题，不替代语义治理，也不绕过风险控制。

分级治理，统一运行时 三个独立维度

统一 BlockPlan 契约
所有业务 · 所有问题

→ **能力来自物料，不来自契约记法**
同一入口 · 同一解析 · 同一编译主链

将原来混在 L1–L3 里的三件事拆开

L1 / L2 / L3
物料 · 算子 · 策略
混成一个运行时等级
拆开

→

物料完备度
指标 · 维度
关联 · 枚举
时间锚点 · 主键
能问什么

编译器算子集
聚合 · 窗口
否定存在 · 递归
能力独立演进
能表达什么

业务风险策略
strict
standard
exploratory
敢不敢直接出数

统一入口，不等于无条件放行

维度缺定义 → 可执行，但标注
指标缺口 → 回显假设，确认后执行

关联未声明 → 拒绝，不推断 JOIN
算子超出 → 受控兜底并标记 L0

已上线：统一契约 · 元素级处理 · 缺口分型 · 引用级保障标注；待持续验证完善

BlockPlan
负责表达用户这次想查什么；

OSI
负责说明这些指标、维度、关联和业务值到底是什么；

元素级解析
负责判断本次实际引用了哪些物料；

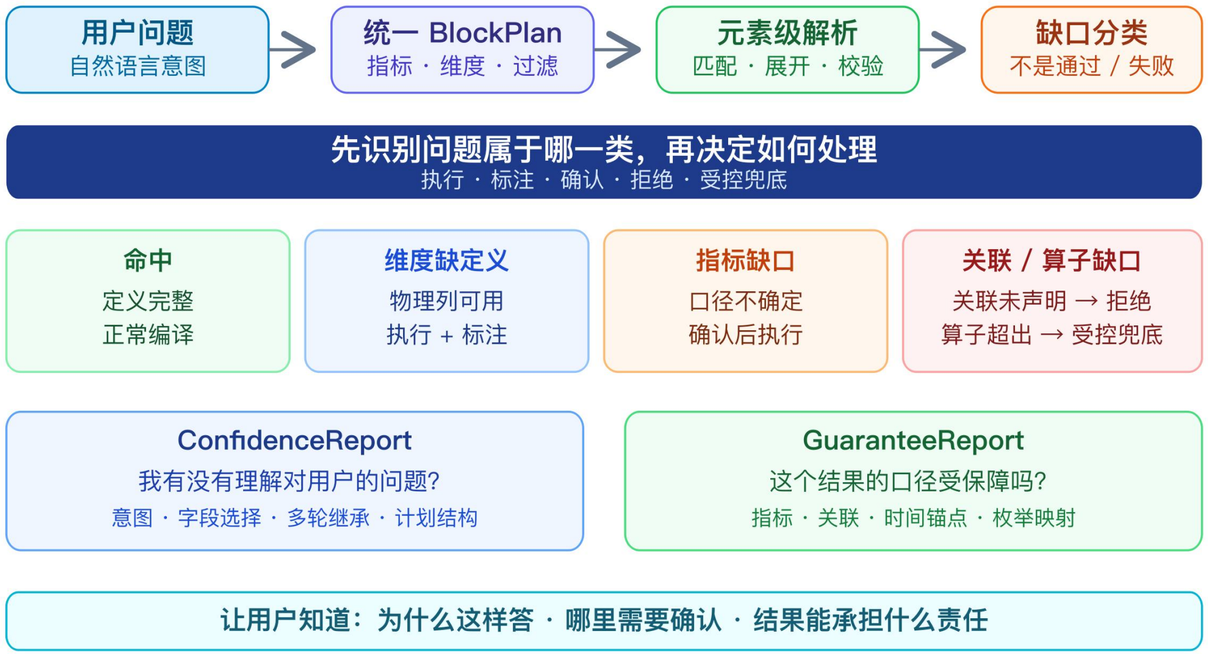
编译器算子集
负责判断当前引擎会不会计算这种问题；

风险策略
负责决定缺口出现时是执行、确认、拒绝还是兜底；

GuaranteeReport
负责说明结果的业务口径有多大保障；

ConfidenceReport
负责说明系统对用户意图理解得有多确定。

面向信任——从“通过/失败”到可解释的结果责任



发现的问题	分类	处理
正式指标、正式维度、已声明关联	hit	正常编译
字段存在，但只有物理信息	raw_dim	可执行，但标注
没有正式指标，只能临时 **SUM(amount)**	ad_hoc_measure	回显假设，确认后执行
用户要求跨表，但没有声明关联	undeclared_join	拒绝
字段和物料都存在，但 **Builder** 不支持窗口操作	unsupported_op	受控兜底或拒绝
引用的字段在 **OSI** 中完全不存在	unknown	走幻觉处理

统一契约负责承载请求，OSI 物料负责提供语义，编译器能力负责执行，风险策略负责兜底边界。

选型第一步：分清三层概念

选型第一步：把三个问题拆开——管指标、管执行、管治理，别混成一个

层	回答的问题	典型承载
指标层（管理）	这个指标什么意思、谁负责、怎么审批变更	指标目录 / OpenMetadata Glossary
指标语义层（执行）	这次查询怎么编译成正确的 Join / 聚合 / 过滤	MetricFlow / Cube Core / 自研编译器
元数据语义层（治理）	术语、资产、标签、血缘、影响分析	OpenMetadata

不同厂商边界不同，本文按职责拆分

开源组件

工具	定位	强项	代价
MetricFlow	dbt 体系指标编译库	复杂指标 / 时间语义 / 定义随 dbt 进 Git	绑定 dbt 工作流，服务化要自建
Cube Core	独立无头语义服务	SQL / REST / GraphQL 多消费端 + 预聚合缓存	要运营一个语义服务（容量 / 缓存 / HA）
Lightdash	语义层 + BI + 目录一体	定义 / 目录 / 探索 / 仪表盘一条流水线	与 Lightdash 使用方式绑定较紧
Apache Superset	BI 内置轻量语义能力	主要服务 Superset 内部分析，外部语义层集成仍在演进	不宜与完整 headless 语义服务等价比较
OpenMetadata	元数据与业务语义治理	术语 / 负责人 / 血缘 / 影响分析	不做指标计算，须与执行层组合

场景速配：dbt 主干 → MetricFlow | 服务多端 / AI Agent → Cube Core | 快速目录 + 自助分析 → Lightdash
仅 Superset、规模小 → 轻量语义层 | 指标算得出但没人负责 → OpenMetadata + 执行层

03 双路径混合路由

双通道语义 × Data Agent 端到端架构

结构化语义主干 · 模糊语义补给 · 三个合流点 · 显式覆盖边界

① 语义供给与治理（离线闭环）

七类语义原料
①库表元数据 ②术语映射 ③ETL血缘
④业务文档 ⑤SQL日志 ⑥BI报表 ⑦运行时

抽取与派生
LLM 批量抽取 · 血缘 DAG 自动派生

结构化语义存储
OSI SemanticModel · 实时拼装

模糊语义存储
弱参考资产库 · 带 provenance
已验证答案 · Golden Queries

人工确认闸 · 转正入库
精度达标才入库 · 降级 SQL 审核转正
P4 硬闸：负收益即停止

② Agent 在线主链

用户问题 + 表清单 + 多轮上下文

结构化通道（强语义）
resolve 拼装 SemanticModel
裁剪 A 关键词 · C 打分 · D 种子+BFS
E 不变式 → 越界整体回退全量
OSI 白名单校验 · 幻觉检测

模糊通道（弱参考）
资产检索：报表·日志·文档·运行时
相关度排序 → 预算裁剪 → 安全净化
输出带 provenance 的弱参考候选
reject-only：只拒绝、不改 SQL

合流点 1 | 术语映射注入检索词典（只提召回） | 模糊语义永不进 AST

路由判决 PreMatch
top-1 打分 + 实词锚定 · 未过则直通模糊路径（省 1 次意图 LLM）

结构化路径
意图解析 LLM（1 次）
BlockPlan 规划 + 白名单校验
5 类 Builder 确定性编译
编译失败 → 降级至模糊路径

模糊路径（fallback）
合流点 2 | 弱参考块注入
业务语义补充 · 历史问答 · 未锚定术语
合法标识符白名单（硬约束）
LLM 直出 SQL

统一防护：安全校验 · EXPLAIN · 执行
自愈：结构化错误三元组 {类型·标识符·相似列} 回喂重试（cap 3）

统一出口
结果 + 置信度 + dispatch_path 标注 + 语义覆盖报告
dropped / bare / unresolved 显式暴露（不静默降级）

③ 运行时支撑与反哺闭环

能力编排
归因 · 下钻 · What-If
分析洞察 · 本体推理 · Runbook

标准接入
MCP Server · REST API · Skill

治理与防线
行列权限 · PII 脱敏 · 成本限额

可观测
全链路 Trace · 语义缓存

缺失语义补偿
运行时探查 · 数据画像
假设-验证环（APEX 式）

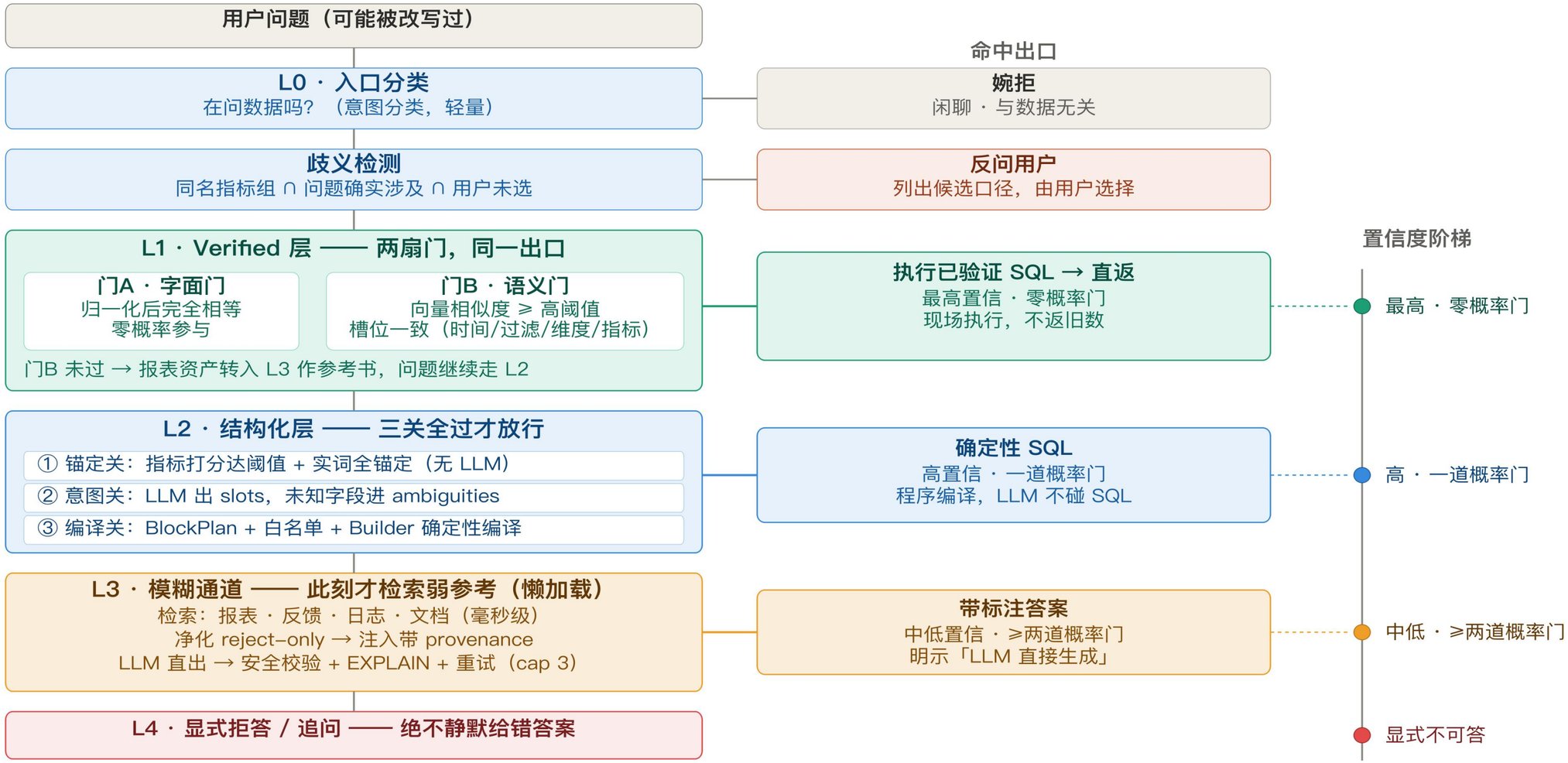
评测与反哺闭环
命中率 · held-out 准确率
口径一致性（同问异写同答）
被采纳 SQL 人工审核 → 转正入库
合流点 3 回流语义层

评测反哺 · 转正入库

铁律：模糊语义永不进 AST · 降级必须可见 · 漏召回必须可逆 · 答不了就别答

用户问题的分层命中逻辑

每往下一层，置信度降一档，且降档必须让用户看见

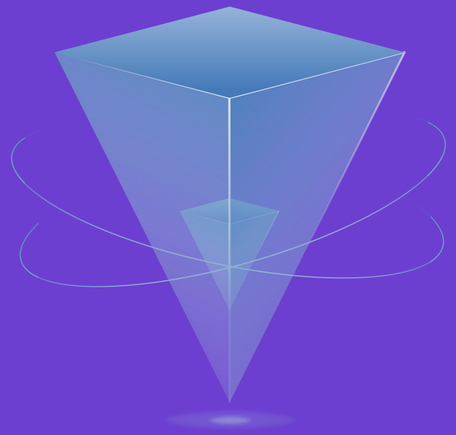


三个必盯指标 + 三条诚实边界

1. 覆盖与正确性：分路径命中率 + 各路径业务正确率 + 合理澄清 / 拒答率

2. 泛化与可维护性：独立留出集 + 语义缺陷诊断 + 维护成本）

3. 一致性与稳定性：等价改写一致 + 固定条件下重复运行稳定



1、可编译 ≠ 业务正确：合法指标也可能选错，Join粒度、过滤和时间口径仍需验证。

2、已建模 ≠ 全覆盖：超出表达或语义范围时，澄清、受控降级或显式拒答。

3、外部高分 ≠ 本地收益：必须在自己的问题、数据、权限和版本条件下重测。

终局不是全覆盖语义层，而是在明确覆盖边界内可靠回答，边界外显式澄清、降级或拒答。

04 持续完善的评测体系

三级评估漏斗

① 测试数据集（三源归一）

- **人工标注黄金集**
  t0_questions.jsonl · 10 条
- **用户点赞反馈集**
  三层过滤 + 四阶采样 · DB 直连
- **异常边界 + 多轮对话**
  anomaly_cases + YAML

② 三种测试模式

- **Mock 框架自检**
- **Live 快速评估**
- **Live+Semantic 精确评估**

③ SQL 执行与结果集比对

pred_sql → pred_rows vs **gold_sql → gold_rows** → compare() → **L1 多集精确比对（Counter）**

④ 三级评估漏斗

- **L1 精确匹配**
  Counter 多集比对
  忽略列序/行序 · 1e-6 容差
- **L2 语义确认**
  结构相似值不同
  → LLM Judge 语义判定
- **L3 不匹配**
  无共享列 / 多出额外列
  → 人工审核

并行校验： ExpectedFacts · Hallucination · SchemaLinking · Calibration · Multi-Turn

⑤ 指标汇总 & 失败归因

- **核心指标**
  EX=(L1+L2)/总数 · 三级占比 · VES 效率分 · ECE 校准
- **失败归因（7 类 AST 对比）**
  table→column→filter→join→agg→syntax→timeout

只看结果集，不看 SQL 文本 | 参考 Databricks Genie Benchmarks
程序能判的不浪费 LLM，LLM 能判的不浪费人

语义层冷启动：多路原材料



冲突检测、来源优先级、有效期、责任人、冲突解决机制

加速器：LLM智能导入——批量从 schema、血缘和文档起草初版语义；低风险描述可抽检，核心指标、Join、粒度和权限必须审核后转正

评测反哺语义层

反馈三层阻断漏斗

用户点击 · 点赞

一：高频异常过滤

二：SQL AST 语法校验

三：AI 审计语义等价

纯净落库

阻断核心原则

宁可漏掉，不可错收

确保进入飞轮的每一条

数据都是高质量资产

四阶自适应采样

① 时间滑动窗口（N 天）

② 语法骨架指纹去重

③ 分层权重采样

AST 0.6： Fallback 0.3： Other 0.1

④ 差额自动补足

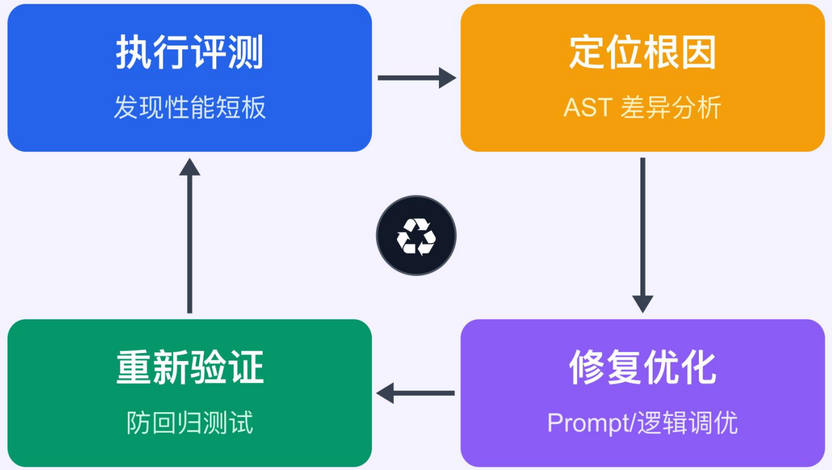
生成代表性回归测试集

采样核心目标

在控制测试成本的前提下

最大化数据分布的多样性

持续改进循环（飞轮引擎）



飞轮效应（Flywheel Effect）

每一次循环都在积累势能

测试集越丰富 → 归因越精准

修复越高效 → 模型越智能

最终实现系统的自动进化

结构性防幻觉纪律与语义层过拟合

	prompt 恳求（概率性防御）	lift-only（结构性防御）
做法	“请只使用真实存在的表名”	标识符不在编译器输出集合 → 拒绝执行
保证	大概率听话，无硬保证	错的标识符物理上到不了数据库
依赖	模型自觉	流程闸门，换什么模型都成立

```
# 第一版（健康的）
description：订购收入，下单口径的销售额

# 打磨到第 N 版（drift 了）
description：订购收入，下单口径的销售额。
                当问题出现"趋势/月度"时选本元素而非 shippedRevenue；
                若提到"净"字需先扣退款… …
```

标识符安全≠语义正确

两块常开仪表：命中率与语义质量

仪表一 | 分层命中率

—— 用户真实问题接住没有

四层分布 · 每条查询落到哪层，有痕迹

L1 Verified 直返 | L2 结构化编译 | L3 LLM 直出 | L4 显式拒答——dispatch_path 逐条落库

语义路径命中率 = L1 + L2 占比

聚合即得：不跑评测、基于已有日志可低成本聚合

一鱼两吃 · 价值证明 + 执行开关

对上：语义层到底有没有用的唯一诚实答案；对内：≥70% 弱参考懒加载，<30% 意图期并行预取

业界锚点

Snowflake官方Routing Mode文档披露的语义SQL路由占比约10%

仪表二 | 语义层质量信号

—— 不依赖下游准确率，地图画得怎么样

四个刻度 · 在语义层自己身上量

缺 synonyms 指标占比 | 孤立 dataset | join 路径覆盖率 | 零引用表占比——模型表直接统计，零成本随时可算

hold-out 题集 · 防「押题」漂移

description 会 drift 成泄漏答案的文本——调优集与留出集分数一对照就现形

出身：open problem → 每天自动跑的工程

SMQ 论文自承「独立于准确率的质量信号」是开放问题——这里把它做成仪表

命中率答「用户的问题接住没有」，质量信号答「地图画得怎么样」

命中率掉了，去质量信号找病因；质量补上了，看命中率涨回来

THANKS

演讲嘉宾：郭朝阳